

Implementation of Peer-to-Peer Energy Trading via Solana Smart Contracts in a Permissioned Environment

Chanthawat Kiriyaadee¹, and Suwannee Adsavakulchai^{1*}

¹Department of Computer Engineering and Artificial Intelligence, School of Engineering, University of the Thai Chamber of Commerce (UTCC), Bangkok, Thailand

Contact Email: 2410717302003@live4.utcc.ac.th, Suwannee_ads@utcc.ac.th

Thailand's three state utilities EGAT, MEA, and PEA have advanced a National Energy Trading Platform (NETP) since 2017 to facilitate Peer-to-Peer (P2P) trading among rooftop-solar prosumers, envisaging a blockchain-based settlement layer with a dedicated digital currency. However, remains in development, and Thailand's regulated buy-back scheme still provides no direct mechanism for prosumers to exchange surplus generation within the local electrical grid. This paper presents a smart-contract-based on-chain settlement layer for P2P energy trading, realized as five modular Anchor programs deployed on a permissioned Solana runtime that utilize the Sealevel parallel-execution engine. By routing every high-frequency write to a per-entity program-derived account (PDA), the system permits transactions with disjoint write sets to execute in parallel, with transaction ordering provided by Solana's Proof-of-History (PoH). The evaluate market clearing price framework couples a continuous double auction (CDA) for real-time trades with a uniform-price auction that clears aggregated prosumer bids into a single settlement price over fifteen-minute epochs. We simulate energy consumption, generation, and power flow for a network of M meters over D days at fifteen-minute intervals within a low-voltage grid topology, of which 15% of meters are prosumers. The full token life-cycle minting, settlement, and burning is evaluated on a single-node validator (Apple M2, Agave 3.1.10). We report the resulting throughput, energy-conservation, and net-revenue outcomes, each independently re-derived and audited from on-chain state.

1. Introduction

Since 2017 Thailand's state utilities have pursued a National Energy Trading Platform (NETP) for P2P trading among rooftop-solar prosumers, implemented on a Tendermint-based consortium blockchain [1]. This article presents a Solana-based architecture as an alternative execution layer, motivated by NETP's own evaluation, in which Tendermint outperformed both Ethereum and Hyperledger Fabric under throughput testing: that result suggests further gains from a runtime whose execution is parallel rather than sequential. Such evaluations report throughput and compute cost as though computation were the axis along which a settlement layer scales. The contribution here is to separate the axes — partition all hot-path state per entity, then measure compute, throughput and latency independently, so that what binds each can be named rather than assumed — and then, in a second and separate experiment, to confirm that the market built on that partition settles correctly and profitably against audited on-chain state.

2. The Per-Entity Partition and Measurement Method

- **Smart-contract layer.** The settlement layer comprises Smart Contract programs [2] (anchor-lang and anchor-spl 1.0.0), written in Rust and compiled to SBF bytecode for the Solana Virtual Machine, in which programs invoke one another through cross-program invocation (CPI), with authority delegated to PDAs that sign without any private key existing.
- **Per-entity partition.** A transaction declares in advance every account it touches and whether each is writable, and the runtime locks per account, so that two transactions with disjoint writable sets execute on different cores whereas two writing one account serialise. Every high-frequency write therefore targets its own PDA — MeterState per meter, Order per order, OrderNullifier per settlement replay guard, and registration counters sharded sixteen ways — while global accounts such as configuration and running totals remain read-only on hot paths and deliberately stale, folded forward by periodic administrative instructions.
- **Environment.** Both experiments were conducted on a single node (Apple M2, Agave 3.1.10) of a private solana-test-validator network, so that they characterise SVM semantics — account locking and compute metering — rather than consensus or leader rotation [3].
- **Metric.** Throughput is reported as client-observed confirmed goodput, defined as confirmed transactions per wall-clock second from burst start to last confirmation, and every non-confirmed transaction is attributed to one of three classes — send-rejected, guard-rejected, or expired — which separates delivery loss from validation operating as intended.
- **Experiment A (scaling sweep).** One reading per meter is submitted from N distinct meters over two epochs at least 61 s apart, covering $N = 80$ to 200,000 (1.35 M transactions).
- **Experiment B (community-month replay).** The full token life-cycle — minting, settlement, and burning — is driven over the month of Table 2, on a topology taken from the CINELDI 80-bus rural LV reference grid [4] and resolved with pandapower [5], with time-series produced by the simulator under a fixed seed and solar modelled via pvlib [6].

The two experiments use different harnesses; their rates are therefore not mutually comparable.

3. Primary Result: Throughput Orders by Lock Footprint

Compute is $O(1)$ and never binds. In Experiment A the steady telemetry write costs 13,337–13,634 CU from 80 to 200,000 meters — the entire 2,500× range spanned by 297 CU ($\pm 1.1\%$) — against a 200,000-CU default per-instruction budget, and the final submissions cost the same against 510,000 resident PDAs as the first scale did against 10,000. The first write for a meter costs 16,050 CU, because it initialises and rent-funds the PDA. Latency is likewise flat, p50 ≈ 1.5 –1.9 s and p95 ≤ 3.1 s across the whole sweep, where a contended design would degrade; delivery loss is zero across 1.02 M first-attempt sends.

Throughput orders by lock footprint, but the mechanism does not confirm. Table 1 sorts every measured path by the shared writable state it touches, and that ordering — not the CU column — tracks the rate across three orders of magnitude. The association is strong; the causal test fails. Anchor’s mut imposes a writability requirement, not a prohibition, so the shipped binary still accepts the pre-fix lock footprint if the client declares ZoneMarket writable — one AccountMeta, nothing else changed. Over five repeats of 1,200 settles, the writable arm returns 493 against 561 confirmed settles·s⁻¹ across eight fee payers (overlapping ranges), and 447 against 446 across one: even the shared fee-payer lock does not move throughput, because conflicting transactions still land within a slot through successive entries, and at ≈120 k CU per settle the block compute budget saturates before locks bind. We therefore withdraw the 2.7–3× previously attributed to deleting this mut; a previously reported ≈0.6 TPS for single-payer settlement reproduces exactly (0.69 TPS, p50 463 ms per settle) when the harness awaits each confirmation before the next send — a measurement-loop property, since the same path sustains ≈450 TPS with transactions concurrently in flight. Order entry carries the same caveat: the **cheaper** instruction (9,884 CU against 13,337) returned 180 TPS against 462 while its context declared the shared zone-market account writable though the handler mutated only the per-shard account; the lock is no longer established as its cause.

Table 1: **Throughput by lock footprint** (Experiment A). Paths ordered by the shared write-locked state they touch — an ordering, not a demonstrated cause (see text). Harnesses differ per row; rows are not mutually comparable.

Path	CU	TPS	Shared writable state
RPC read	—	≈1,454	none — no consensus round-trip
meter ingest (5 k–200 k meters)	13.3–13.6 k	426–490	gateway fee payer
order entry (10 k orders, pre-fix)	9,884	180	fee payer + zone_market
OLTP proxy (concurrency 5 → 40)	22–24 k	7.9 → 74.3	fee payer + market accounts
settlement, awaited serially (harness-bound)	≈115 k	≈0.6	— (one block-time RTT per settle)
settlement, concurrent, 1–8 fee payers	≈120 k	446–561	fee payer + energy_mint + collectors

3.1. Secondary Result: Audited Community-Month Settlement

Experiment B replays the month of Table 2 in 1,394.7 s (1,858× real time) with zero delivery loss; all 919 rejected readings are deterministic anomaly-gate rejections. Under this harness telemetry sustains ≈213 readings·s⁻¹ and order bursts ≈53 TPS, and each Ed25519-verified settlement costs 107 k CU. The energy-closure identities hold exactly on chain — 15 of 15 audit assertions — under all three price rules, and net revenue orders uniform 2.290 > buy-back 2.200 > CDA 2.065 THB/kWh, so wheeling policy sets the P2P participation threshold.

Table 2: **The simulated community-month dataset** (Experiment B; seed-deterministic, no field data).

Quantity	Value
fleet	80 meters: 12 prosumer sellers, 68 consumers
market policy	sell cap $C = 10$ kWh/day per prosumer; 0.1 kWh dust floor
horizon / readings	30 days × 96 ticks ($\Delta t = 15$ min) = 230,400 readings, integer Wh
month energy	15,868.5 kWh generated; 144,879.8 kWh consumed; 10,386.7 kWh interval surplus
oracle-accepted surplus	8,253.502 kWh; 919 readings gate-rejected

4. Conclusion

Over the range measured here, partitioning hot-path state per entity leaves neither computation nor per-entity contention as the binding constraint: the steady write varies by ±1.1% from 80 to 200,000 meters and against 510,000 resident accounts, and confirmation latency does not degrade across that sweep. What orders the measured paths is lock footprint on the few genuinely shared accounts, more closely than instruction cost does — but that ordering did not survive a controlled test as a mechanism: with the pre-fix lock footprint reproduced on the shipped binary, settlement throughput is unchanged from one fee payer to eight, and the two strongest prior data points dissolve under re-measurement, one into validator warm-up, one into a harness that awaited each settle’s confirmation before sending the next. Shrinking declared write sets remains sound design; on a single node it buys nothing measurable here, and the order-entry path has not been re-measured since the same change was applied to it. The community-month replay is a seeded simulation rather than field data, and within it the energy-closure identities held in all 15 audit assertions and P2P net revenue under uniform-price clearing exceeded the regulated buy-back rate. These results characterise one implementation on a single validator, which bounds execution and locking only; reproducing them on a multi-validator consortium of the form NETP itself prototyped across EGAT, MEA, and PEA [1] would place consensus under test, and suits UEC–ASEAN collaboration.

Acknowledgment

The authors thank the University of the Thai Chamber of Commerce (UTCC) for institutional support, and the organizers, the UEC ASEAN Research Center (UARC) and Multimedia University (MMU), for hosting this workshop.

References

- [1] Metropolitan Electricity Authority (MEA) and Electricity Generating Authority of Thailand (EGAT) and Provincial Electricity Authority (PEA), “Development of the National Energy Trading Platform (NETP): Research and Development Project Report and Digital Platform Roadmap for Thailand’s Electricity Authorities,” technical report, Dec. 2019.
- [2] Coral / Anchor contributors, “Anchor: Solana’s Sealevel runtime framework.” [Online]. Available: <https://www.anchor-lang.com/>
- [3] Solana Foundation, “Tower BFT: Solana’s High Performance Implementation of PBFT.” Jul. 2019.
- [4] L. M. Engan, S. Ekrheim, S. Bjarghov, J. Klemets, G. Kjølle, and I. Schytte, *Reference dataset for semi-urban and rural Norwegian low voltage distribution system*. (Dec. 2024). Zenodo. doi: 10.5281/zenodo.14528192.
- [5] L. Thurner *et al.*, “pandapower—An Open-Source Python Tool for Convenient Modeling, Analysis, and Optimization of Electric Power Systems,” *IEEE Transactions on Power Systems*, vol. 33, no. 6, pp. 6510–6521, 2018.
- [6] W. F. Holmgren, C. W. Hansen, and M. A. Mikofski, “pvlib python: A python package for modeling solar energy systems,” *Journal of Open Source Software*, vol. 3, no. 29, p. 884, 2018, doi: 10.21105/joss.00884.